

TactiVision Pro

Code Ownership Split

Technical Interview Prep

Project Code: DS07

Misr International University

Team of 6

Prepared by: Mohanad (Team Lead)

2026-04-16

How to Use This Document

The technical interview is in 2 days. Each member owns a set of files and must be able to explain them in detail. Use this document as your personal study guide for your work stream. Read your section - know every function, design decision, and push state.

- Your section lists all files under your ownership.
- 'State' column: committed = already on GitHub; modified = tracked but changed locally; UNTRACKED = new file not yet on GitHub; MISSING = blueprint requires it but it does not exist yet.
- 'Push?' column: YES = stage and push to your feature branch; Study = read and understand, already on GitHub; Decide / Coordinate = talk to the team lead.
- Shared files (main.py, config.yaml, services/__init__.py, database_schema.py, database_manager.py) must go through Mohanad.

Snapshot of Repository State

Already on GitHub (8 commits)

- Root: main.py, config.yaml, README*.md, CHANGELOG.md, CONTRIBUTING.md, TESTING.md, integrate_to_database.py
- services/: __init__, advanced_analytics, ai_tactical_recommendations, ball_detector, ball_tracker, database, database_manager, database_schema, event_detector, highlights_generator, match_analytics_integration, pitch_transform, player_performance_analytics, possession_tracker, sprint_detector, tactical_analyzer, tracking_filters, xg_calculator
- demo/: dashboard_final.py, demo_analytics.py, run_demo.py, simple_tracker.py
- tests/: test_ball_tracker.py, test_possession.py, test_possession_validation.py

Still to push - new files and modifications

- 15 new service files, 7 new test files, rosters/, data/, scripts/, matches.db, yolov8n.* - see per-member breakdown below.

DO NOT push (should be in .gitignore)

- matches.db - runtime SQLite data
- yolov8n.pt, yolov8n.onnx - large model binaries
- input_videos/*.mp4 - gigabytes of video
- demo/demo_outputs/arsenal/heatmap_*.png - hundreds of generated images
- TactiVision_Pro_Feature_Portfolio.docx, feature_report.js, _ul, 1 - stray artifacts

CORE ENGINE - Mohanad (Team Lead)

Owns the pipeline backbone, batch/chunked processing, integration, and all shared files. Every merge goes through him.

Key responsibilities

- Chunked video processing pipeline (§3.1) - 45+ min videos without crash, flat memory, stable track IDs.
- Batch match processing system (§3.2) - scan a folder of videos, process each end-to-end into SQLite.
- Integration & architecture oversight (§3.3) - daily merges, code review, conflict resolution.
- Cross-chunk ID stitching via DeduplicatorV2.cross_chunk_id_matching (position + jersey color, 100px threshold).

Files owned

File	State	Push?
main.py	modified	Yes
config.yaml	modified	Yes - shared
requirements.txt	modified	Yes
services/__init__.py	modified	Yes - shared
services/match_analytics_integration.py	committed	Study
services/batch_processor.py	UNTRACKED	Yes
services/tracking_filters.py	modified	Yes
services/ball_tracker.py	modified	Yes
services/ball_detector.py	modified	Yes
services/event_detector.py	modified	Yes
services/possession_tracker.py	modified	Yes
services/sprint_detector.py	modified	Yes
services/pitch_transform.py	modified	Yes
integrate_to_database.py	committed	Study
services/formation_detector.py	UNTRACKED	Decide
services/offside_detector.py	UNTRACKED	Decide
services/dribble_detector.py	UNTRACKED	Decide
services/set_piece_detector.py	UNTRACKED	Decide
services/tactical_insights.py	UNTRACKED	Decide
services/opponent_analyzer.py	UNTRACKED	Decide

Interview talking points

- How chunked processing keeps memory flat (gc.collect + torch.cuda.empty_cache between chunks).
- Why cross-chunk ID stitching uses position + jersey color rather than appearance embeddings.
- BatchProcessor isolation: a crash in match 3 must not affect match 4.
- Which files are SHARED and require team lead review before merge.

DASHBOARD & UI - Samy

Owns the entire Streamlit dashboard: error hardening, match selector, comparison tab, and visual polish.

Key responsibilities

- Dashboard error hardening (§4.1) - every tab loads without errors, match selector at the top, defensive null/empty data handling.
- Match comparison tab (§4.2) - side-by-side analytics for 2-5 matches, comparison table + grouped bars + radar chart.
- Dashboard visual polish (§4.3) - team colors, st.metric cards on Overview, spinners, consistent plotly palette.

Files owned

File	State	Push?
demo/dashboard_final.py	modified	Yes - primary file
demo/dashboard_fotmob.py	UNTRACKED	Decide
demo/dashboard.py (deletion)	deleted	Yes - stage deletion

Interview talking points

- How st.session_state propagates the selected match_id to all 10 tabs.
- Defensive pattern: 'if data is None or len(data) == 0: st.info(...); return'.
- Radar chart normalization (0-100) across Shots/Passes/Possession/Sprints/xG.
- Why all 10 tabs are wrapped in try/except, not just the fragile ones.

ROSTER SYSTEM - Helal

Owns replacing the static JSON roster files with a database-backed system, including schema, service layer, and UI.

Key responsibilities

- Database roster schema & migration (§5.1) - teams, roster_players, match_rosters tables; migration logic.
- Roster manager service & JSON migration (§5.2) - services/roster_manager.py with import_from_json, get_player_lookup, create_team_with_players.
- Dashboard roster management UI (§5.3) - create/edit team, import JSON, assign roster to match.
- Update services/player_identity.py to try DB first, fall back to JSON with deprecation warning.

Files owned

File	State	Push?
services/player_identity.py	UNTRACKED	Yes
services/player_profile_system.py	UNTRACKED	Yes (if used)
services/database_manager.py	modified (roster CRUD)	Yes - co-own w/ Mohanad
services/database_schema.py	committed (roster tables)	Co-own - explain schema
rosters/liverpoolvsbournemouth.json	UNTRACKED	Yes
rosters/liverpoolvscity.json	UNTRACKED	Yes
rosters/test.json	UNTRACKED	Yes
services/roster_manager.py	MISSING	Create before interview

Interview talking points

- Schema design: why match_rosters maps matches to home/away team_ids rather than duplicating player rows.
- Backward-compat fallback chain: DB -> JSON -> warn-and-continue.
- Why CRUD uses parameterized queries (no f-strings) - SQL injection safety.
- How get_roster_lookup feeds {jersey_number: player_name} into the analytics pipeline.

QA & TESTING - Seif

Owns measurable accuracy: ground-truth annotation, validator, regression testing across every sprint day.

Key responsibilities

- Ground truth annotation (§6.1) - annotate 15+ min of one match with every pass/shot/sprint in JSON.
- Accuracy validation script (§6.2) - AccuracyValidator with match_events, calculate_metrics (precision/recall/F1), report.
- Daily regression testing (§6.3) - pytest every day, flag new failures to Mohanad.

Files owned

File	State	Push?
tests/ground_truth/	UNTRACKED	Yes
tests/accuracy_reports/	UNTRACKED	Yes
tests/test_accuracy_measurement.py	UNTRACKED	Yes
tests/ACCURACY_TESTING_GUIDE.md	UNTRACKED	Yes
tests/test_suite.py	UNTRACKED	Yes
tests/test_ai_tactical_recommendations.py	UNTRACKED	Yes
tests/test_player_performance_analytics.py	UNTRACKED	Yes
tests/test_ball_tracker.py	committed	Study
tests/test_possession.py	committed	Study
tests/test_possession_validation.py	committed	Study

Interview talking points

- Precision = $TP / (TP + FP)$; Recall = $TP / (TP + FN)$; F1 = harmonic mean. Division-by-zero returns 0.0.
- Why a 3-second time tolerance - detections rarely align to the exact frame.
- Each GT event matches at most one detection (closest first) to prevent double-counting.
- Where overall weighted F1 comes from - weights = event count per category.

DATA & API - Ahmed Khaled

Supplies real match videos and live squad data via the football-data.org API connector with caching and rate limiting.

Key responsibilities

- Source 3-5 match videos (§7.1), 15+ min each, broadcast angle, delivered to input_videos/.
- Football-Data.org API connector (§7.2) - fetch_squad, fetch_and_save_squad, 6-second rate-limit sleep, JSON cache.
- Pre-fetch rosters for demo matches (§7.3) into the database so the demo works without live API.

Files owned

File	State	Push?
services/api_connector.py	UNTRACKED	Yes
data/squads_top5.json	UNTRACKED	Yes (API cache)
config.yaml (api section)	modified by Mohanad	Coordinate
input_videos/*.mp4	UNTRACKED (arsenal.mp4 deleted)	NO - .gitignore
rosters/*.json (API-generated)	UNTRACKED	Yes - hand-off to Helal

Interview talking points

- Why the free tier rate limit (10 req/min) forces a 6-second sleep between calls.
- Cache strategy: {team_name_slug}_{endpoint}.json - check cache before hitting the API.
- Graceful failure modes: timeout, 429, 404 - all must return clearly without crashing the pipeline.
- Naming convention: league_teamA_vs_teamB_date.mp4.

DOCS & DEMO - Youssef

Owns the end-to-end demo, PDF match report generator, cached demo bundle, and documentation updates.

Key responsibilities

- Demo script (§8.1) - demo/run_demo.py covering 6 stages (DB init > roster > process > stats > PDF > dashboard) in <10 min.
- Pre-cached demo data bundle (§8.2) - matches.db + JSON summaries in demo/demo_data/ so demo runs without a GPU.
- PDF match report generator (§8.3) - services/report_generator.py via fpdf2; 2-4 pages.
- README update & presentation outline (§8.4).

Files owned

File	State	Push?
demo/run_demo.py	modified	Yes - primary file
services/video_exporter.py	UNTRACKED	Yes
services/social_export.py	UNTRACKED	Yes (if used)
services/report_generator.py	MISSING	Create before interview
README.md	committed	Update
CHANGELOG.md	committed	Update
TESTING.md	committed	Update
README_ANALYTICS.md	committed	Update
demo/simple_tracker.py	committed	Study
demo/demo_analytics.py	committed	Study

Interview talking points

- 6-stage flow and where each stage reads/writes from the DB.
- --no-cache vs default cached mode - why cached is the default for laptop demos.
- PDF sections: header, summary, key stats table, event timeline, formation summary, top performers.
- How demo data bundle lets the demo run without re-processing video.

Gaps to Address Before the Interview

Files the blueprint requires but do not exist yet

- services/roster_manager.py - Helal, §5.2
- services/report_generator.py - Youssef, §8.3

Decision: create stubs before the interview, or present these as the natural next step in the remaining 20% roadmap.

Shared-file rule (blueprint §2.2)

- No one modifies main.py, config.yaml, __init__.py, or database_schema.py without telling Mohanad first.
- Mohanad performs all merges to shared files.
- Branch naming: feature/[stream]-[task] (e.g. feature/roster-db-schema, feature/dashboard-comparison-tab).

Git workflow for the next 2 days

- Each member: checkout -b feature/your-stream, push your untracked files, open a PR tagging Mohanad.
- Mohanad reviews and merges in the order listed in §3.3.1 daily integration duties.
- Run python -m pytest tests/ before every push - no red tests on main.

Next Steps After This Document

- Fill code gaps: create roster_manager.py and report_generator.py stubs.
- Clean the repo with .gitignore so matches.db, videos, and heatmaps stop clogging commits.
- Build the presentation outline (Youssef leads, all members contribute their stream's highlight).
- Final demo rehearsal the night before the interview.